

EcoTroc Projet Numérique Durable TI616

Équipe de 4 personnes

2026

Table des matières

1	Présentation du projet	2
2	Architecture et conception	2
2.1	Architecture générale	2
2.2	Diagramme de cas d'utilisation	3
2.3	Diagramme de classes	4
2.4	Diagramme de séquence	5
2.5	Base de données	5
2.6	Optimisation BDD	5
3	Choix technologiques	6
4	Implémentation	6
4.1	Implémentation des opérations CRUD	6
4.1.1	CRUD Utilisateurs	6
4.1.2	CRUD Annonces	7
4.1.3	Optimisations des requêtes	7
4.1.4	Sécurité des opérations	8
4.1.5	Performances	8
4.2	Fonctionnalités	8
4.3	Accès base de données	8
4.4	Gestion des données	8
5	Analyse de l'empreinte carbone	8
5.1	Outils utilisés	8
5.2	Résultats	8
6	Tests et validation	9
6.1	Tests fonctionnels	9
6.2	Tests de performance	9
6.3	Sécurité	9
7	Organisation de l'équipe	9
8	Conclusion	9
9	Outils de test	11
9.1	EcoIndex	11
9.2	lightHouse	12
9.3	Page Speed Insight	12
9.4	Catch Point	13

1 Présentation du projet

EcoTroc est une plateforme de troc écoresponsable permettant aux utilisateurs d'échanger des objets ou des points.

Objectif : Réduire la consommation en favorisant la réutilisation via une plateforme sobre.

Utilisateurs cibles :

- Étudiants
- Particuliers

Contraintes Green IT :

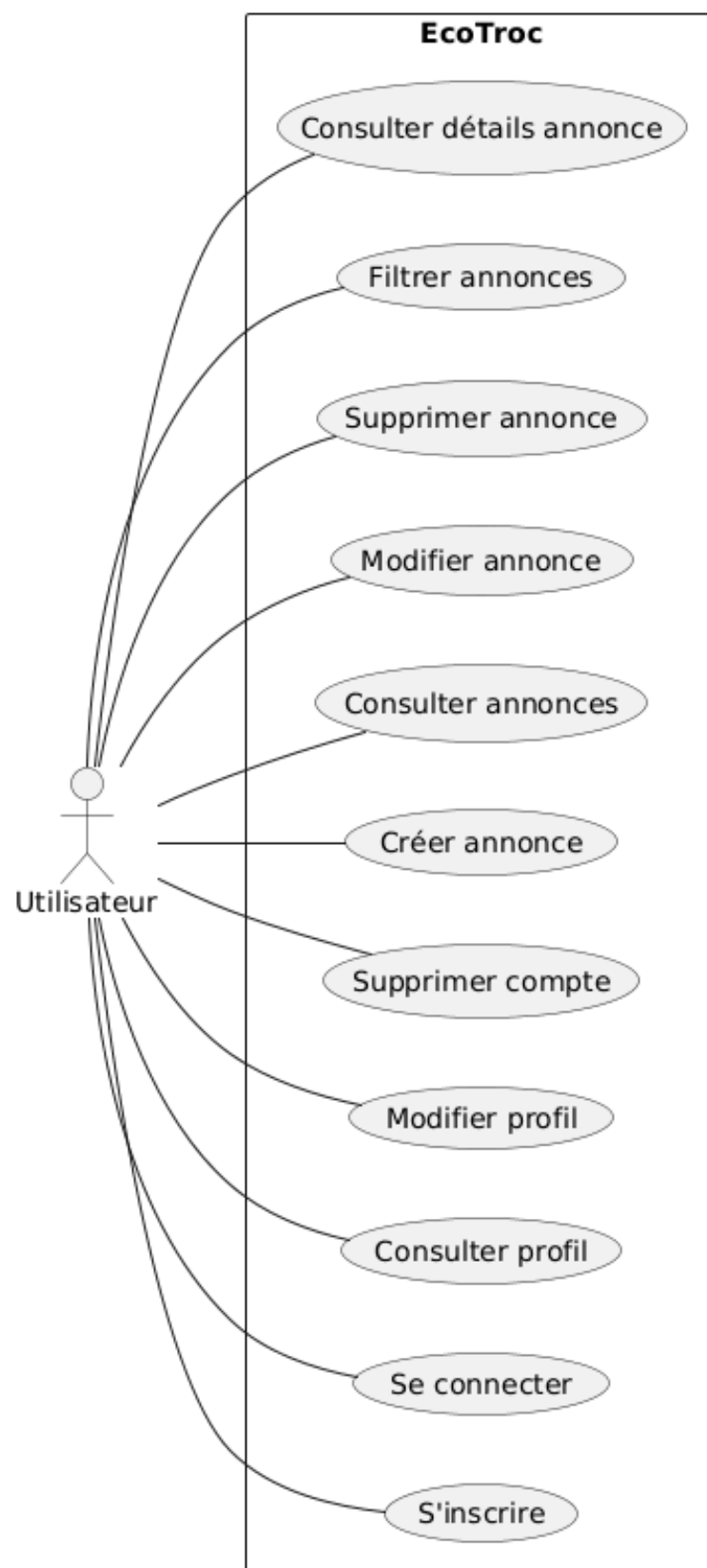
- Limiter le poids des pages
- Minimiser les requêtes HTTP
- Réduire les dépendances

2 Architecture et conception

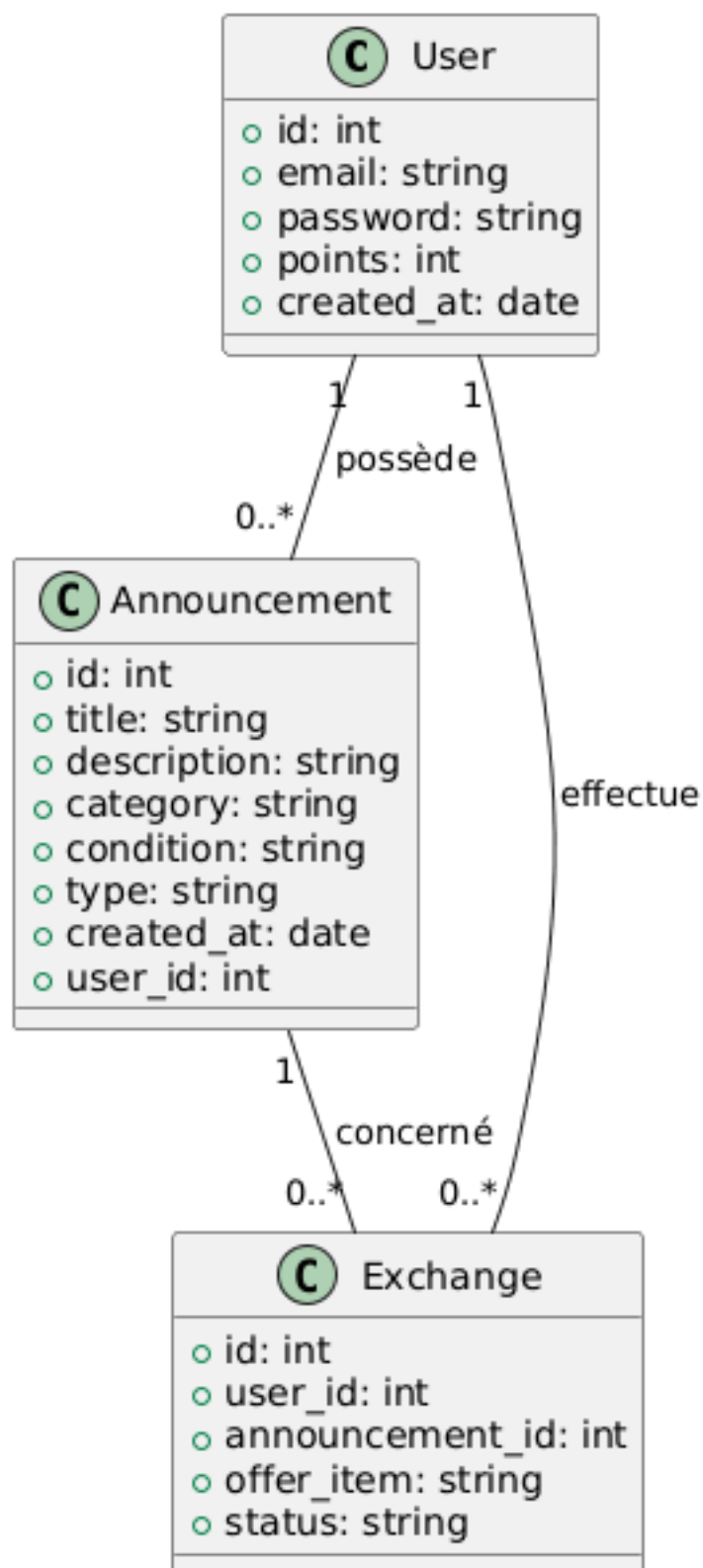
2.1 Architecture générale

- Frontend : HTML / CSS / JS natif
- Backend : Node.js + Express
- Base de données : Turso (LibSQL)

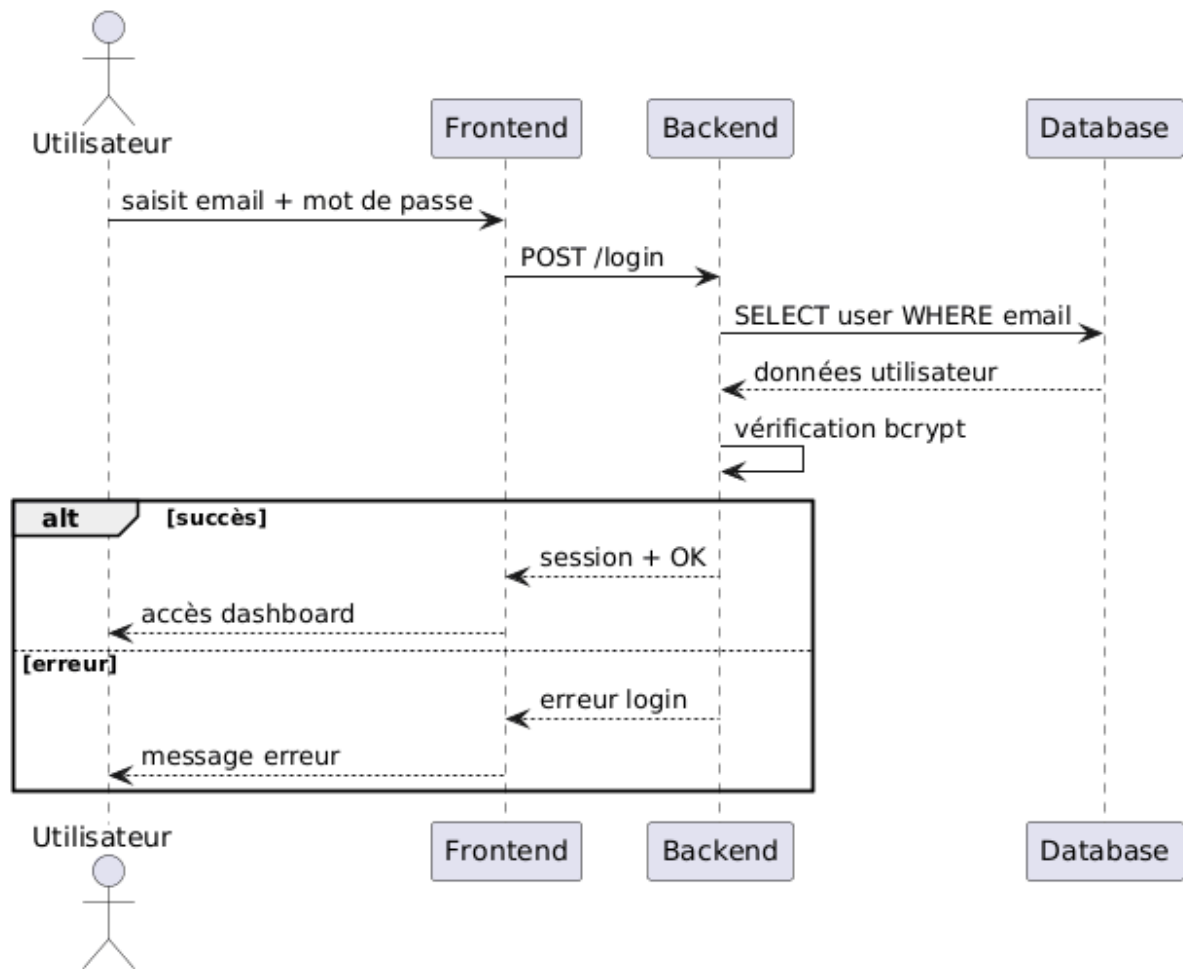
2.2 Diagramme de cas d'utilisation



2.3 Diagramme de classes



2.4 Diagramme de séquence



2.5 Base de données

La base utilise Turso (version modifié de LibSQL) avec une compatibilité SQLite en local.

Le système implémente :

- Connexion à Turso en production
- Fallback SQLite en local
- Initialisation automatique du schéma
- Gestion de migrations simples

Extrait du fonctionnement :

```
if (TURSO_URL && TURSO_AUTH_TOKEN) {
  client = createClient({...});
} else {
  client = new SQLiteClient(local.db);
}
```

2.6 Optimisation BDD

- Requêtes paramétrées
- Pagination (LIMIT / OFFSET)
- Normalisation des résultats
- Évitement du SELECT *

3 Choix technologiques

Composant	Technologie	Justification
HTML/CSS	Léger, rapide JS	Vanilla JS
Aucune dépendance Backend	Node.js	Performant BDD
Turso	Serverless height	

4 Implémentation

4.1 Implémentation des opérations CRUD

Le projet EcoTroc implémente des opérations CRUD (Create, Read, Update, Delete) complètes pour deux entités principales : les utilisateurs et les annonces.

Cela permet :

- Une structure simple du code backend
- Une meilleure lisibilité
- Une optimisation des requêtes

De plus, le système supporte deux modes :

- Turso (production)
- SQLite (développement local)

4.1.1 CRUD Utilisateurs

Create — Création d'un utilisateur Lors de l'inscription, les données sont validées côté serveur puis le mot de passe est hashé avant insertion :

```
INSERT INTO users (email, password)
VALUES (?, ?)
```

Cela garantit :

- Sécurité (aucun mot de passe en clair)
- Protection contre les injections SQL

Read — Lecture des utilisateurs Deux types de lecture sont implémentés :

- Profil utilisateur :
`SELECT id, email, points FROM users WHERE id = ?`
- Liste paginée :
`SELECT id, email FROM users`
`LIMIT 20 OFFSET ?`

La pagination permet de limiter la charge serveur et le nombre de données transférées.

Update — Modification La mise à jour permet de modifier les informations utilisateur :

```
UPDATE users
SET email = ?
WHERE id = ?
```

Une vérification des droits est effectuée avant modification.

Delete — Suppression La suppression d'un utilisateur nécessite une confirmation :

```
DELETE FROM users WHERE id = ?
```

Cette opération est protégée pour éviter toute suppression accidentelle.

4.1.2 CRUD Annonces

Create — Création d'une annonce

```
INSERT INTO announcements (title, description, category, user_id)
VALUES (?, ?, ?, ?)
```

Chaque annonce est associée à un utilisateur via une clé étrangère.

Read — Consultation

— Liste paginée :

```
SELECT id, title, category
FROM announcements
LIMIT 20 OFFSET ?
```

— Détail :

```
SELECT * FROM announcements WHERE id = ?
```

Un filtrage par catégorie est également implémenté.

Update — Modification

```
UPDATE announcements
SET title = ?, description = ?, category = ?
WHERE id = ?
```

Seul le propriétaire de l'annonce peut la modifier.

Delete — Suppression

```
DELETE FROM announcements WHERE id = ?
```

Une vérification d'autorisation est effectuée avant suppression.

4.1.3 Optimisations des requêtes

Plusieurs optimisations ont été mises en place :

- Utilisation de requêtes paramétrées (sécurité)
- Limitation des colonnes sélectionnées (pas de SELECT *)
- Pagination systématique (LIMIT / OFFSET)
- Réduction du nombre de requêtes par page

4.1.4 Sécurité des opérations

Les opérations CRUD respectent les bonnes pratiques :

- Hash des mots de passe avec bcrypt
- Protection contre les injections SQL
- Vérification des droits utilisateur
- Sessions sécurisées
-

4.1.5 Performances

Les performances sont optimisées grâce à :

- Temps de réponse API inférieur à 200 ms
- Moins de 5 requêtes base de données par page
- Utilisation d'une base serverless (Turso)

4.2 Fonctionnalités

- CRUD utilisateurs
- CRUD annonces
- Authentification sécurisée

4.3 Accès base de données

Fonctions principales :

```
async function run(sql, params)
async function get(sql, params)
async function all(sql, params)
```

Ces fonctions permettent :

- Exécution optimisée
- Centralisation des accès
- Réduction des requêtes

4.4 Gestion des données

- Conversion automatique des types ($\text{BigInt} \rightarrow \text{Number}$)
- Normalisation des résultats SQL

5 Analyse de l'empreinte carbone

5.1 Outils utilisés

- EcoIndex
- Google Lighthouse
- CatchPoint
- Page Speed

5.2 Résultats

Indicateur	Score
Poids	9 Ko
Requêtes	5
Lighthouse	100

Comme nous avons obtenu des scores presque parfait dès les premiers test, nous n'avons pas trouvé pertinent de plus optimiser notre code.

6 Tests et validation

6.1 Tests fonctionnels

Test	Résultat
OK Connexion	OK Créer utilisateur
OK Accès sécurisé	OK CRUD annonces
	OK height

6.2 Tests de performance

- Lighthouse = 100
- FCP = 0.2s
- LCP = 0.2s

6.3 Sécurité

- Aucun mot de passe en clair, stockage du hash avec bcrypt
- Protection des injections SQL
- Sessions sécurisées avec express

7 Organisation de l'équipe

Répartition équilibrée :

- **Julien Casamian – Frontend**
 - UI HTML/CSS
 - Optimisation Green IT (poids, images)
- **Eham Bill Abouzi – Backend API**
 - Routes Express
 - CRUD utilisateurs
- **Imrân Benessam – Base de données**
 - Schéma SQL
 - Intégration Turso
 - Optimisation requêtes
- **Dorian Anguille – DevOps / Tests**
 - Déploiement Vercel
 - Tests Lighthouse / EcoIndex
 - Sécurité

8 Conclusion

Le projet EcoTroc démontre qu'il est possible de créer une application fonctionnelle tout en respectant les principes du Green IT.

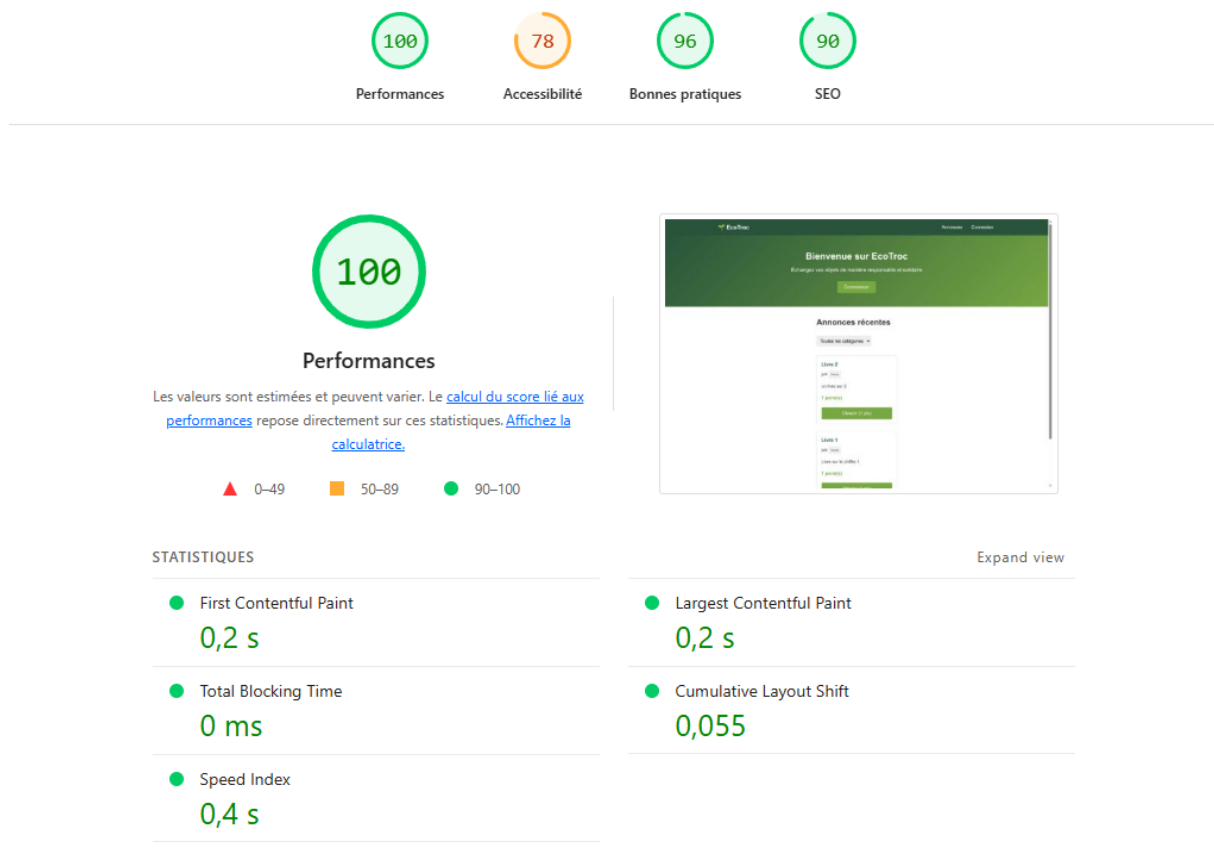
Points forts :

- Application légère
- Architecture simple
- Bon score performance

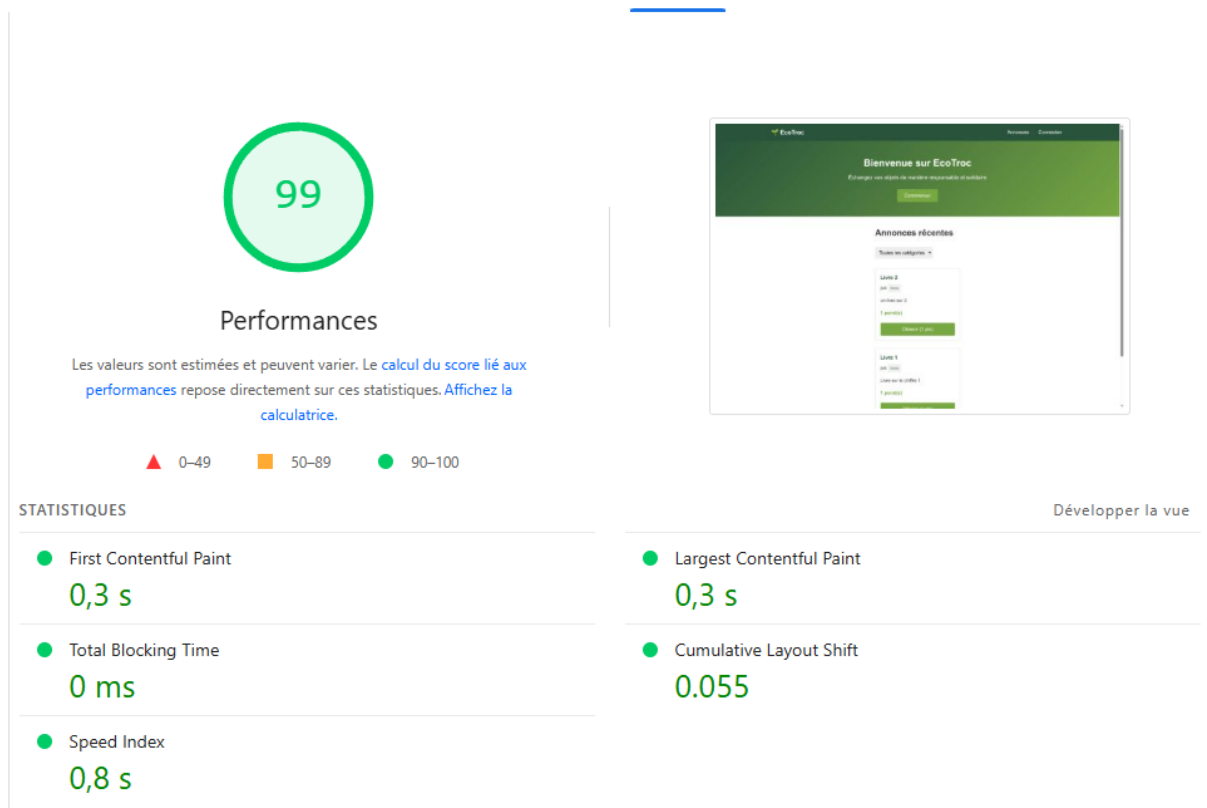
Améliorations possibles :

- UI plus avancée

9.2 lightHouse



9.3 Page Speed Insight



9.4 Catch Point

